

Introduction

Every automation tool eventually teaches you the same lesson: the bugs that hurt the most are the ones that don't crash. A typo throws an error. A missing field throws an error. But a hardcoded value that's *technically valid*, just wrong? That runs quietly, forever, until someone notices the numbers don't add up.

This is exactly the failure mode n8n's date variables are designed to prevent, and it's worth understanding deeply — not because \$now and \$today are exotic APIs, but because the underlying pattern (dynamic values vs. typed-in values) shows up in every automation platform, every ETL pipeline, and every cron job you'll ever maintain. If you're building or maintaining n8n workflows professionally, this is one of the first things that separates a workflow that survives six months from one that quietly corrupts data in week three.

We'll also cover two related skills that pay off just as often: using n8n's built-in string, array, and object methods instead of reaching for a Function node, and replacing single-decision IF nodes with a ternary expression. None of this requires regex or custom JavaScript beyond a one-liner — that's the whole point.

Problem Overview

Picture a workflow with a date typed directly into a field: `2026-08-11`. It works today, because today is 2026-08-11. Tomorrow, it's still there, unchanged, now describing a day that already passed. Nothing in the workflow errors out. It just filters, compares, or reports against the wrong date, and whatever depends on that data — a report, a query, a conditional branch — silently drifts out of sync with reality.

This is the "time bomb" pattern: a value that is correct at the moment it's written and incorrect at every moment after. The fix isn't "remember to update it" — humans forget, and automation exists precisely so humans don't have to remember things. The fix is to replace the static value with an expression that recalculates itself every time the workflow runs.

n8n solves this with two built-in variables, \$now and \$today, plus a broader expression system for strings, arrays, objects, and conditionals that lets you avoid hardcoding almost everything else too.

Example

Say a workflow needs to tag today's orders for a daily report.

Hardcoded version:

```
filterDate = "2026-08-11"
```

Runs correctly on August 11. On August 12, it still filters for August 11 — yesterday's orders, silently, forever.

Dynamic version:

```
filterDate = {{ $today }}
```

`$today` re-evaluates on every execution. On August 12 it returns August 12 at midnight. The workflow never needs to be touched again.

Now consider a second field that needs the *exact* run time, not just the date — say, a timestamp for an audit log:

```
runTimestamp = {{ $now }}
```

If you swapped this for `$today` instead, every log entry from a given day would show midnight, regardless of when the workflow actually ran. That's a different, subtler bug: technically dynamic, but still wrong for the use case.

Intuition

The core insight is one question, asked before you touch the keyboard: **is this value ever supposed to change?** If yes, it doesn't belong as literal text in a field — it belongs in an expression that computes it fresh each run.

Once you accept that, the choice between `$now` and `$today` stops being a memorization problem and becomes a comprehension check. Ask what you're actually comparing:

- "What day is it?" → you want `$today` (midnight-stamped, so date comparisons line up cleanly).
- "What exact moment is it?" → you want `$now` (full timestamp, down to the second).

The bug that bites most people isn't picking the wrong one out of ignorance — it's comparing a full timestamp against a midnight-stamped value (or vice versa) and getting a mismatch that looks like a logic error but is actually a granularity error. Once you know both variables exist and what each one is stamped to, that class of bug disappears.

The same "does this ever change?" question extends past dates. Text that needs cleaning, a nested field from an API response, a single yes/no decision — all of these are candidates for inline expressions instead of hardcoded values or extra nodes.

Brute Force Solution

The "brute force" approach here isn't algorithmic — it's architectural: solve every dynamic-value problem with a dedicated node.

- **Idea:** Use a Function node to run custom JavaScript for any text cleanup, date math, or transformation. Use an IF node for every conditional decision, however small.
- **Algorithm:** For each place a value needs to be computed or branched, add a canvas node, wire it in, and write the logic inside it.
- **Advantages:** Very explicit — anyone can see a full JavaScript function or a two-branch IF node and understand it without knowing expression syntax.
- **Disadvantages:** Canvas bloat. A workflow that could be five nodes becomes fifteen. Every extra node is another thing to maintain, another point of failure, and another click required to understand a one-line decision. It also doesn't prevent the hardcoded-date bug at all — a Function node with a hardcoded date inside it has the exact same time-bomb problem as a field with a hardcoded date.
- **Time/space complexity:** Not applicable in the algorithmic sense, but in workflow terms: O(n) extra nodes for n small decisions, and O(n) extra maintenance surface.

Optimal Solution

The optimal approach uses n8n's expression system directly, inline, in the field that needs the value:

Need	Brute force	Optimal
Current date	Hardcoded string	<code>{{ \$today }}</code>
Current timestamp	Hardcoded string	<code>{{ \$now }}</code>
Trim whitespace	Function node	<code>{{ \$json.field.trim() }}</code>
Normalize casing	Function node	<code>{{ \$json.field.toLowerCase() }}</code>
Check substring	Function node	<code>{{ \$json.field.includes("x") }}</code>
Transform each item	Function node	<code>{{ \$json.items.map(...) }}</code>
Filter items	Function node	<code>{{ \$json.items.filter(...) }}</code>
Nested field access	Function node with parsing	<code>{{ \$json.data.user.email }}</code>

Need	Brute force	Optimal
Yes/no decision	IF node, two branches	<code>{{ \$json.status === "done" ? "Complete" : "Pending" }}</code>

The mental model: n8n gives every field a small, sandboxed JavaScript expression context. `$json` is the current item's data, `$node` reaches into another node's output by name, and `$workflow` exposes metadata like the workflow ID. Combined with standard JavaScript string/array/object methods, most single-value logic collapses into one line in one field — no separate node, no separate wire, no separate thing to debug.

For the ternary specifically: an IF node is "check a condition, go left or go right" drawn as a shape. A ternary expresses the identical logic — `condition ? valueIfTrue : valueIfFalse` — as text in a single field. Same two branches, zero extra nodes.

Python Code

While n8n expressions run in JavaScript, the underlying pattern — dynamic values, string/collection methods instead of custom logic, and conditional expressions instead of branching structures — maps directly onto Python. Here's the equivalent logic, useful if you're porting this workflow's decision logic into a Python-based automation or writing a test harness against n8n's webhook output:

```
from datetime import datetime, date

def get_now() -> datetime:
    """Equivalent of n8n's $now: exact current timestamp."""
    return datetime.now()

def get_today() -> date:
    """Equivalent of n8n's $today: midnight-stamped current date."""
    return date.today()

def normalize_status(item: dict) -> str:
    """Clean text without a dedicated processing step, same idea as
    n8n's inline .trim().toLowerCase().includes()."""
    status = item.get("status", "").strip().lower()
    return status

def status_label(item: dict) -> str:
    """Ternary in place of an IF/branch structure."""
    return "Complete" if normalize_status(item) == "done" else "Pending"
```

```
def filter_by_today(items: list[dict], date_field: str) -> list[dict]:
    """Compare against $today equivalent, not $now, to avoid the
    granularity mismatch bug."""
    today = get_today()
    return [
        item for item in items
        if datetime.fromisoformat(item[date_field]).date() == today
    ]
```

Code Walkthrough

- `get_now()` returns a full `datetime` — the Python analogue of `$now`, precise to the second.
- `get_today()` returns a `date` object with no time component — the analogue of `$today`, already "midnight-stamped" by virtue of the type itself.
- `normalize_status()` mirrors the inline string methods from the video: `.strip()` is `.trim()`, `.lower()` is `.toLowerCase()`. No separate cleaning step, no separate function — just the transformation applied where the value is used.
- `status_label()` replaces a branching if/else block with a single conditional expression, the same trade n8n makes when swapping an IF node for a ternary field.
- `filter_by_today()` is the guard against the exact bug in the video: it explicitly compares `.date()` to `.date()`, so a full timestamp is never accidentally compared against a midnight-stamped value.

Dry Run

Input `items`:

```
[
  {"status": "Done", "created_at": "2026-08-11T14:32:00"},
  {"status": "pending", "created_at": "2026-08-10T09:00:00"},
]
```

Running on 2026-08-11:

1. `get_today()` → `date(2026, 8, 11)`.
2. `filter_by_today(items, "created_at")` keeps only the first item, since `2026-08-11T14:32:00` truncates to `2026-08-11`, matching today; the second item's `2026-08-10` doesn't match.
3. `normalize_status(items[0])` → `"done"` (stripped and lowercased).
4. `status_label(items[0])` → `"Complete"`.

Run the same code tomorrow, unchanged: `get_today()` recalculates to `2026-08-12`, and the filter automatically follows — no manual update required. That's the entire fix for the hardcoded-date bug, expressed as code.

Complexity Analysis

- `get_now()` / `get_today()` : $O(1)$ time, $O(1)$ space — a single system call.
- `normalize_status()` / `status_label()` : $O(k)$ time where k is string length, $O(k)$ space for the normalized copy.
- `filter_by_today()` : $O(n)$ time where n is the number of items (one comparison each), $O(m)$ space for the filtered result, where $m \leq n$.

These are correct because none of the operations require looking at more than one item at a time or holding more state than the current pass requires — there's no nested loop or repeated re-scanning anywhere in the logic.

Alternative Solutions

An alternative to inline expressions is a dedicated "config" node or variable that stores computed dates once at workflow start, then references that stored value throughout. This can be reasonable in workflows with many nodes needing the *same* timestamp, since it guarantees consistency (every node sees the identical instant) and can slightly reduce redundant computation. The trade-off is an extra node and a layer of indirection for a benefit that's usually negligible unless the workflow is large and performance-sensitive.

Edge Cases

- **Empty or missing date field:** `$json.field` returns undefined; guard with `$json.field?.trim() ?? ""` rather than letting the expression throw.
- **Timezone mismatches:** `$now` and `$today` use the workflow's configured timezone; comparing against externally-sourced UTC timestamps without normalizing first reproduces the same class of bug as `$now` vs `$today` confusion.
- **Midnight boundary runs:** A workflow triggered at 23:59:59 and one at 00:00:01 the next minute will see different `$today` values one second apart — expected, but worth accounting for in reports that span that boundary.
- **Duplicate items in arrays:** `.filter()` and `.map()` don't deduplicate; if uniqueness matters, that's a separate, explicit step.

- **Very large item arrays:** Inline expressions still run once per item in n8n's execution model; for very large batches, a Function node processing the whole array at once can outperform many small inline evaluations.

Common Mistakes

1. Hardcoding any date, ID, or environment-specific value directly into a field instead of deriving it from an expression.
2. Using \$now where \$today is needed (or vice versa), causing off-by-time-of-day mismatches in comparisons.
3. Reaching for a Function node out of habit for a transformation that a single inline string/array method already covers.
4. Building a full IF node for a decision that only maps one condition to one of two output values.
5. Comparing dates as strings instead of parsed date objects, which breaks on inconsistent formatting (e.g., "8/11/2026" vs "2026-08-11").
6. Forgetting that expressions run per-item in a batch, leading to assumptions that a value computed once applies to the whole batch.

Interview Questions

- Why might a system that "never crashes" still be producing wrong output? Walk through how you'd detect that.
- What's the difference between comparing timestamps and comparing dates, and where does that distinction typically cause bugs?
- When would you choose a ternary/inline conditional over a branching structure, and when would you not?
- How would you design a workflow (or codebase) to make hardcoded configuration values harder to introduce by accident?
- Given a batch of records with timestamps, how would you efficiently filter to only those from "today," accounting for timezone?

Similar Problems

- **Valid Date / Date Parsing problems (LeetCode-style):** Any problem requiring you to normalize or compare dates in different formats reinforces the granularity awareness this bug is built on.

- **String to Integer / Data Cleaning problems:** Practicing `.trim()`, `.toLowerCase()`, and validation logic mirrors the inline string-method habit from this lesson.
- **Design a Rate Limiter / Design a Cache with TTL:** Both require reasoning precisely about "current time" vs. "current day," the same distinction `$now` vs `$today` teaches.
- **Ternary/Conditional Expression refactoring exercises:** Common in code review katas — replacing verbose if/else with expressions where it improves readability without sacrificing clarity.

Key Takeaways

- Any value that can change over time does not belong typed in by hand — it belongs in an expression.
- `$now` gives an exact timestamp; `$today` gives a midnight-stamped date. Pick based on the question you're actually asking: "what moment?" vs. "what day?"
- Inline string, array, and object methods (`.trim()`, `.map()`, `.filter()`, dot access) eliminate most reasons to reach for a Function node.
- A ternary expression replaces a full IF node for any single yes/no decision mapping to two output values.
- The underlying principle — dynamic values over hardcoded ones — applies well beyond n8n, to any system with configuration, scheduling, or comparison logic.

Watch the Video

For the live walkthrough, including the real workflow with the hardcoded date being fixed in real time, watch the full video here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series focused on building real, working automations and understanding *why* they behave the way they do — not just clicking through a tutorial, but internalizing the underlying logic so it transfers to the next workflow, the next tool, and the next bug.

Call To Action

If this saved you from a bug you didn't know you had, the like button takes about three seconds and helps the channel a lot more than it costs you. Subscribe on YouTube for the rest of the n8n series, and

subscribe to the Substack for deeper written breakdowns like this one. Drop a comment with any workflow patterns you'd like covered next, and share this with anyone on your team still typing dates into fields by hand.

Quick Review

Difference between \$now and \$today in n8n expressions?

\$now is current DateTime with time; \$today is same date but truncated to midnight (00:00:00).

How to check length of a string/array in an n8n expression?

Use .length like a JS method, e.g. {{\$json.name.length}} — no Function node needed.

Trick to avoid a multi-branch IF node for simple conditional values?

Use a ternary: {{condition ? valueA : valueB}} inline in the expression field.

Common bug ('time bomb') with \$today comparisons?

Comparing \$today against a stored timestamp with a time component always mismatches; normalize both to midnight first.

How do you access nested object/array fields in expressions?

Same dot/bracket JS syntax: \$json.items[0].price or \$json.user.email — treat data as plain JS.

Why avoid overloading one IF node with many unrelated conditions?

It becomes unreadable and hard to debug; split into focused IF nodes or use a ternary per field instead.

n8n Lesson 8: Expressions deep dive (\$now, \$today, string/array/object methods, ternaries, and built-in variables) — free Python cheat sheet